Comparative Analysis of Path Finding Algorithms Under Varying Movement Models and Heuristic Functions in Static Environments

Foreword

THIS PAGE INTENTIONALLY LEFT BLANK.

TABLE OF CONTENTS

Researcher's Note I

This research was conducted with autonomous systems' navigation in mind as underlying all the funatrics, they use pathfinding algorithms to make movements in their various environments. People who'll benefit from this research study are fellow undergraduates, and also graduate students looking to do something related to pathfinding algorithms in a way, like in Robotics, Game AI development, GPS navigation or Network routing.

I've divided it into sections to ease reading because of how lengthy it ended up to be. Section 1 covers the introductory part, telling the reader what problems we currently face with pathfinding algorithms *[see 1.2]* and what we aim to achieve with this study. We'll also take a good look at other comparative studies conducted in the past by fellow researchers, which will be in Section 2.

To not make this paper lenghty than it already is, code snippets for algorithms and statistics calculations will be shared in Jupyter Notebooks with GitHub links provided.

At the time of conducting this study, I was a third-year student at the University of Imo State, Nigeria, studying Computer Science.

For corrections, suggestions, and peer-reviewing kindly reach out to me by email at ebubeakukwe@gmail.com or on LinkedIn "Ebube Akukwe".

THIS PAGE INTENTIONALLY LEFT BLANK.

List of Figures

List of Tables

THIS PAGE INTENTIONALLY LEFT BLANK.

Comparative Analysis of Path Finding Algorithms Under Varying Movement Models and Heuristic Functions in Static Environments

by

Ebube Akukwe

Undergraduate Student in Computer Science

Imo State University, Owerri

2026

Abstract — Efficient pathfinding is important in autonomous systems and robotics. This study compares the performance of BFS, A*, Dijkstra and D* Lite algorithms in static environments. Using simulated grid environments, performance was evaluated based on execution time, path length, and memory usage. Results showed that heuristic-based algorithms like A* and D* Lite consistently required less execution time while producing optimal paths comparable to BFS and Dijkstra. The study concludes that A* is more suitable for real-time navigation.

Section 1 — Introduction

1.1 — Background of the study

When asked what algorithms are, the optimal response will be to say that an algorithm is a step-by-step instruction given to a system, computer, or agent to solve a particular problem. And when such a problem has to do with finding the shortest, cheapest or best path needed to get from one position A to another position B in an environment, pathfinding algorithms will be used.

Pathfinding algorithms are used in various aspects of engineering and computer science, like in robotics, GPS navigation, autonomous systems, network routing and also in game development. But for the purpose of this research we're going to be focusing our attention on simulated navigation based on static, dynamic grids and navgraph environments.

We'll look at the following Pathfinding algorithms,

I. BFS (Breadth-First Search) — Algorithm for finding the shortest path
II. Dijkstra — A cheapest path algorithm with a heuristic function value of 0
III. A*— A best path algorithm with a custom heuristic function value
IV. D* Lite — A path algorithm specifically for dynamic environments

and compare them under different movement models, heuristic functions and environments (static or dynamic), comparing their runtimes, memory usage, nodes explored, path cost, success rate, and replanning cost.

Note: Different algorithms will perform differently depending on their environment (navgraphs or cell-based grids) and other variables, and the "best path" usually depends on the problem being looked at, and the best path may not necessarily be the theoritical shortest path between two nodes.

1.2 — Problem Statement

Although numerous studies have compared pathfinding algorithms, many evaluations focus primarily on identifying the shortest path or minimizing execution time in static environments. Such studies often emphasize algorithmic optimality while giving limited attention to how movement models, heuristic functions, and environmental characteristics such as obstacle density collectively influence algorithm performance. Furthermore, comparisons are frequently conducted using a single movement model or heuristic function, making it difficult to understand how these factors affect search efficiency, node expansions, execution time, and path optimality across different navigation scenarios. Consequently, there remains a need for a systematic

comparative analysis that evaluates pathfinding algorithms under varying movement constraints, heuristic strategies, and environmental configurations within static environments.
This study addresses this gap by comparing selected pathfinding algorithms across different movement models, heuristic functions, and obstacle densities using multiple performance metrics, including execution time, node expansions, and path cost.

1.3 — Aim and Objectives

Aim: To rigorously evaluate and compare the performance of selected pathfinding algorithms under varying movement models, heuristics and environments.

Objectives:
Implement selected pathfinding algorithms and,

I. Evaluate their performance in static environments
II. Compare their execution times, node expansion, peak memory usage and path optimality
III. Analyze the impact of 4 and 8 directional movement model
IV. Analyze the effect of heuristic functions
V. Analyze the effect of obstacle density on search performance

1.4 — Research Questions

I. Which algorithm produces the shortest path most consistently?
II. How do movement models affect algorithm efficiency?
III. Which heuristic provides the best trade-off between speed and optimality?
IV. How do dynamic obstacles affect algorithm performance?
V. Are there relationships between an algorithm's performance and the choice of heuristic, movement model, or environment?

1.5 — Significance of the Study

I. **In Robotics:** This study helps identify algorithms that enable autonomous robots to navigate efficiently through static environments, improving navigation accuracy, obstacle avoidance and energy efficiency.

II. **In Game Development:** The findings could assist game developers in selecting appropriate path-finding algorithms for non-player characters(NPCs), resulting in more intelligent movement, smoother gameplay and reduced computational overhead.

III. **In Intelligent Navigation Systems:** The study supports the development of navigation systems for autonomous vehicles, drones, and mobile robots by evaluating algorithms that provide reliable and efficient route planning.

IV. **Logistics and Warehouse Automation:** The results in this study can guide the selection of path-planning techniques for automated guided vehicles (AGVs) and warehouse robots, improving operational efficiency and minimizing travel time.

V. **Artificial Intelligence Research:** This study could serve as a reference for students like myself investigating search algorithms, heuristic optimization, and decision-making strategies for intelligent agents.

1.6 — Scope of the Study

Algorithms

I. Breadth-First Search (BFS)
II. Dijkstra
III. A*
IV. D* Lite

Movement Models

I. 4-direction
II. 8-direction

Heuristics

I. Manhattan Distance
II. Euclidean Distance
III. Chebyshev Distance
IV. Octile Distance

- **Environments**
  I. Static Environment

- **Variables**
  - algorithm
  - runtime
  - nodes expanded
  - path cost
  - memory
  - grid size
  - movement model
  - heuristic

- obstacle density

THIS PAGE INTENTIONALLY LEFT BLANK.

Section 2 — Literature Review

2.1 — Graph Theory Foundations

The idea of graphs is simply to find a way to represent a position or location (goal), and the possible path or paths (ways) taken to get to those locations. Note that pathfinding problems are commonly modeled using graph theory, where an environment is represented as a graph G consisting of a set of vertices or nodes V, and connecting them are the edges E.

$$G = (V, E)$$

In this representation, nodes correspond to locations or positions, while edges represent possible movements between those locations (West, 2002).
Graph-based representations provide a mathematical framework for analyzing connectivity, reachability, and shortest-path problems. A pathfinding algorithm seeks a path from a start node to a destination node while trying to minimize the number of steps or minimizing a specified cost function. Depending on the application, edge costs may represent physical distance, travel time, energy consumption, or other metrics (Cormen, 2009).

The graph abstraction enables systematic evaluation of search algorithms and provides a consistent basis for comparing algorithmic performance across different environments and problem scenarios (West, 2002).

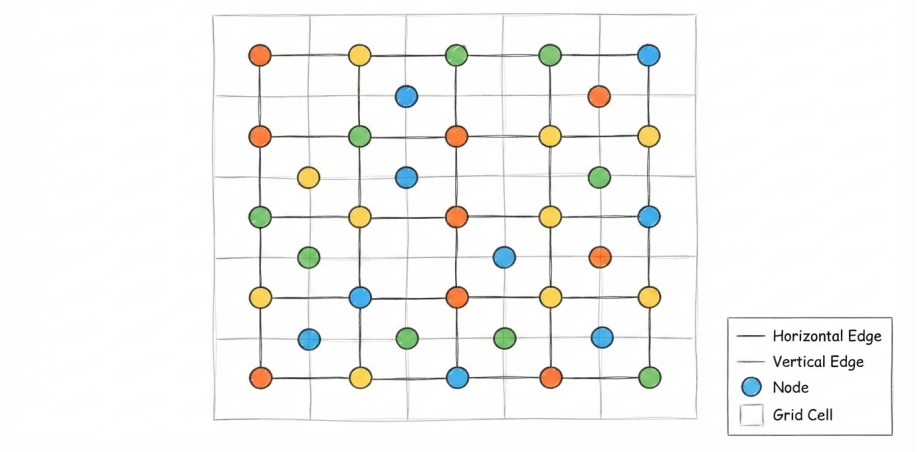

Fig 2.1: Grid environment represented as a graph showing nodes and edges.

2.2 — Pathfinding Algorithms

Although Pathfinding algorithms in general are designed to determine feasible paths between source and destination nodes within a graph, different algorithms may employ different search strategies, resulting in different computational accuracy and quality in solutions.

I. Breadth-First Search (BFS)

Breadth-First Search explores nodes level by level from the starting node. When all edge costs are identical, BFS guarantees discovery of the shortest path because nodes are expanded in order of increasing distance from the source (Cormen, 2009).

II. Dijkstra's Algorithm

Dijkstra's algorithm computes optimal shortest paths in weighted graphs with non-negative edge costs. The algorithm repeatedly expands the node with the lowest cumulative path cost until the destination is reached, guaranteeing optimality under the stated conditions (Cormen, 2009).

III. A* Algorithm

The A* algorithm extends cost-based search by incorporating a heuristic estimate of the remaining distance to the goal. It evaluates nodes using the function

$$f(n) = g(n) + h(n)$$

Where $g(n)$ represents the cost from the start node to current node, and $h(n)$ estimates the remaining cost to the goal.

When the heuristic is admissible and consistent, A* guarantees an optimal solution while typically expanding fewer nodes than uninformed search methods (Hart, Nilsson & Raphael, 1968; Russell & Norvig, 2010).

IV. D* Lite

D* Lite is an incremental heuristic search algorithm developed specifically for dynamic environments. Rather than recomputing an entire path whenever environmental changes occur, it efficiently repairs previously computed paths, thereby reducing computational overhead during replanning (Koenig & Likhachev, 2002).

Although we'll be looking at static environments, in cases where the state of the environment is hybrid (both static and dynamic), we'll also take a good look at how an algorithm like D* Lite developed specifically for dynamic environment would perform when used in a static environment.

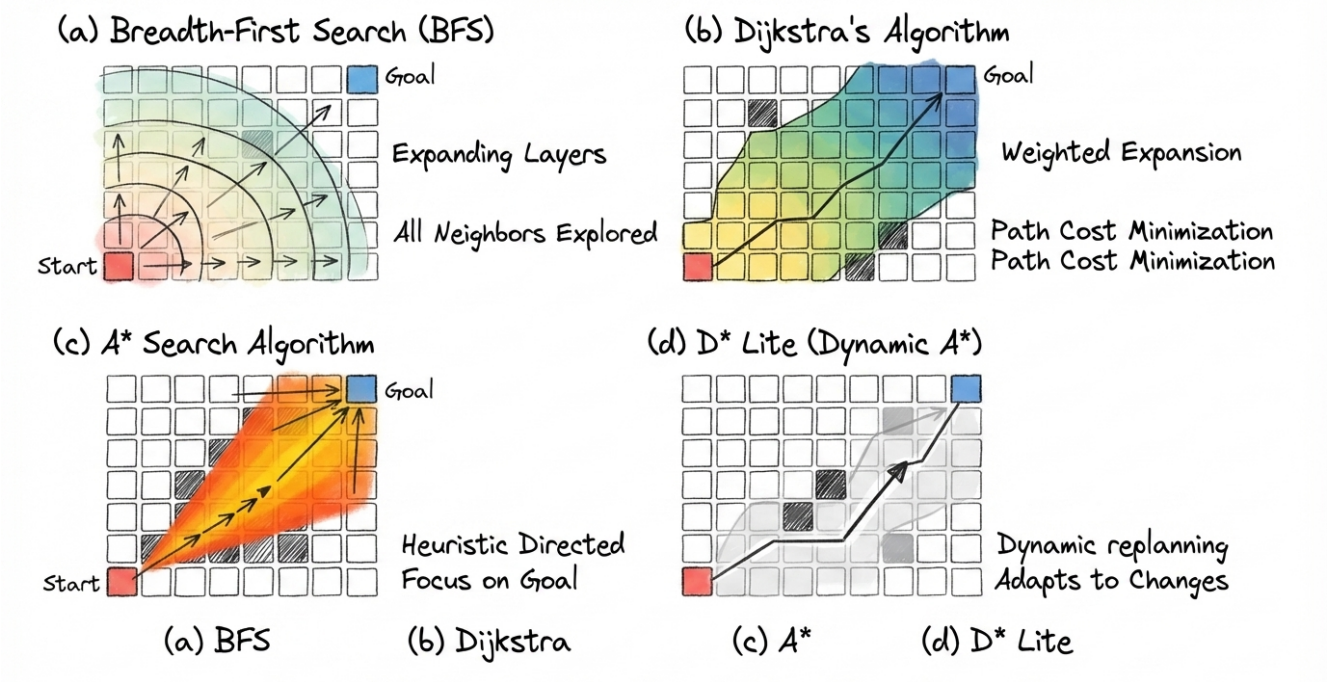

Fig 2.2: *Search progression comparison of BFS, Dijkstra, A*, and D* Lite.*

Existing studies commonly evaluate these algorithms using metrics such as path optimality, execution time, memory consumption, and node expansions (Millington & Funge, 2016).

2.3 — Heuristic Functions

Heuristics are simple problem-specific estimation functions that are used to approximate just how much from the current node is left to reach the destination node or goal. Effective heuristics guide search algorithms toward promising regions of the search space, thereby reducing unnecessary node expansions and improving computational efficiency (Russell & Norvig, 2010).

Common heuristic functions include:

I. Manhattan Distance

The Manhattan distance heuristic is commonly used in environments that allow only horizontal and vertical movements. It is computed as

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

and represents the total number of grid steps required given that diagonal movements are not allowed (Buckland, 2005).

II. Euclidean Distance

Euclidean distance represents the straight-line distance between two points and is suitable when movement can occur in any direction, that is, horizontal, vertical and diagonal. It is computed as

$$h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

(Buckland, 2005).

III. Chebyshev Distance

Chebyshev distance is frequently used in eight-directional movement environments where diagonal and orthogonal movements have equal cost. It is computed as

$$h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$$

and corresponds to the minimum number of moves required to reach the goal under such movement constraints (Buckland, 2005).

The choice of heuristic significantly influences search efficiency and can substantially reduce the number of nodes expanded during search (Hart, Nilsson & Raphael, 1968).

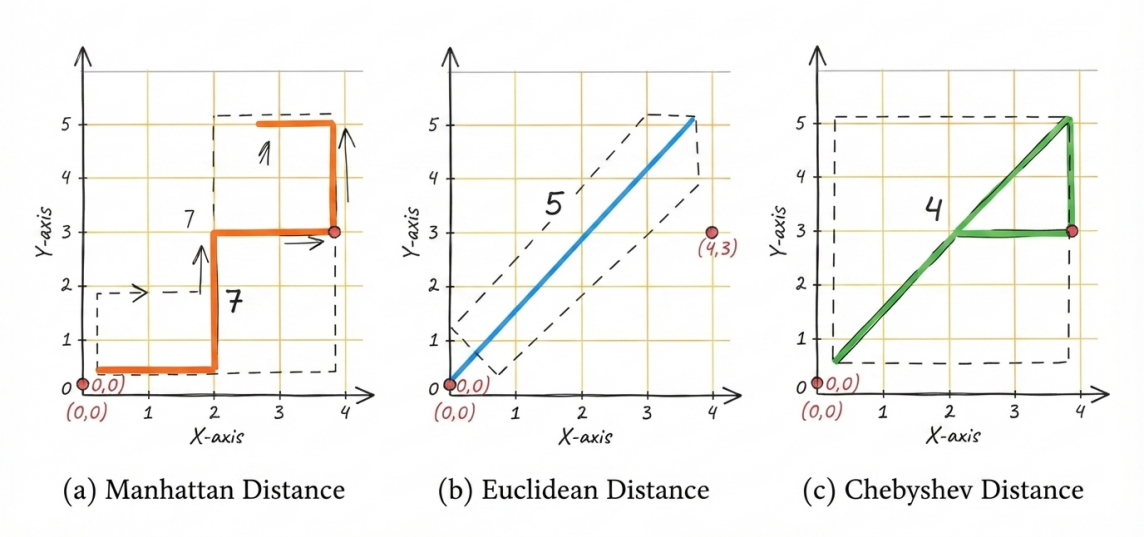

Fig 2.3: Geometric illustration of Manhattan, Euclidean, and Chebyshev distances.

2.4 — Movement Models

Movement models define the allowable actions that an agent may perform while navigating an environment (Millington & Funge, 2016).

I. Four-Directional Movement

In four-directional movement models, agents are restricted to horizontal and vertical movements. This representation is common in grid-based navigation systems and many classical pathfinding applications (Buckland, 2005).

II. Eight-Directional Movement

Eight-directional movement extends the action space by permitting diagonal movements in addition to horizontal and vertical movements. This model often produces shorter paths and more realistic navigation behavior (Millington & Funge, 2016).

Movement models directly influence:

- Path length
- Search space characteristics
- Heuristic selection
- Computational complexity

Consequently, selecting an appropriate movement model is essential when evaluating and comparing pathfinding algorithms (Millington & Funge, 2016).

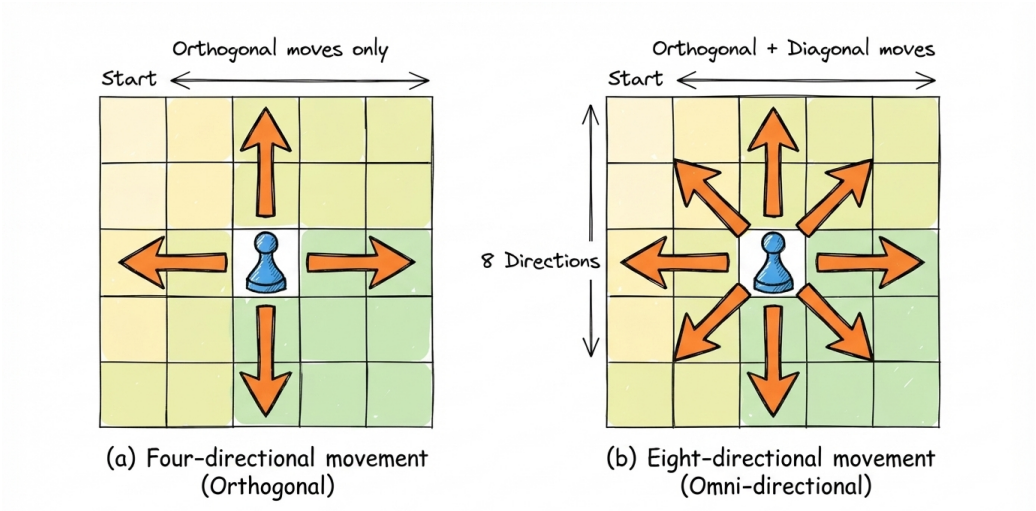

Fig 2.4: Comparison of four-directional and eight-directional movement.

2.5 — Dynamic Environments

Many real-world navigation problems occur in environments that change over time due to moving obstacles, blocked routes, or newly discovered information (Koenig & Likhachev, 2002).

Dynamic environments introduce several challenges, including:

- Frequent replanning requirements
- Increased computational overhead
- Reduced predictability of optimal routes

Algorithms designed primarily for static environments may exhibit reduced performance when environmental conditions change continuously. Consequently, adaptive algorithms such as D* Lite have become increasingly important in robotics, autonomous vehicles, and intelligent navigation systems because they can efficiently update paths without restarting the search process from scratch (Koenig & Likhachev, 2002; Russell & Norvig, 2010).

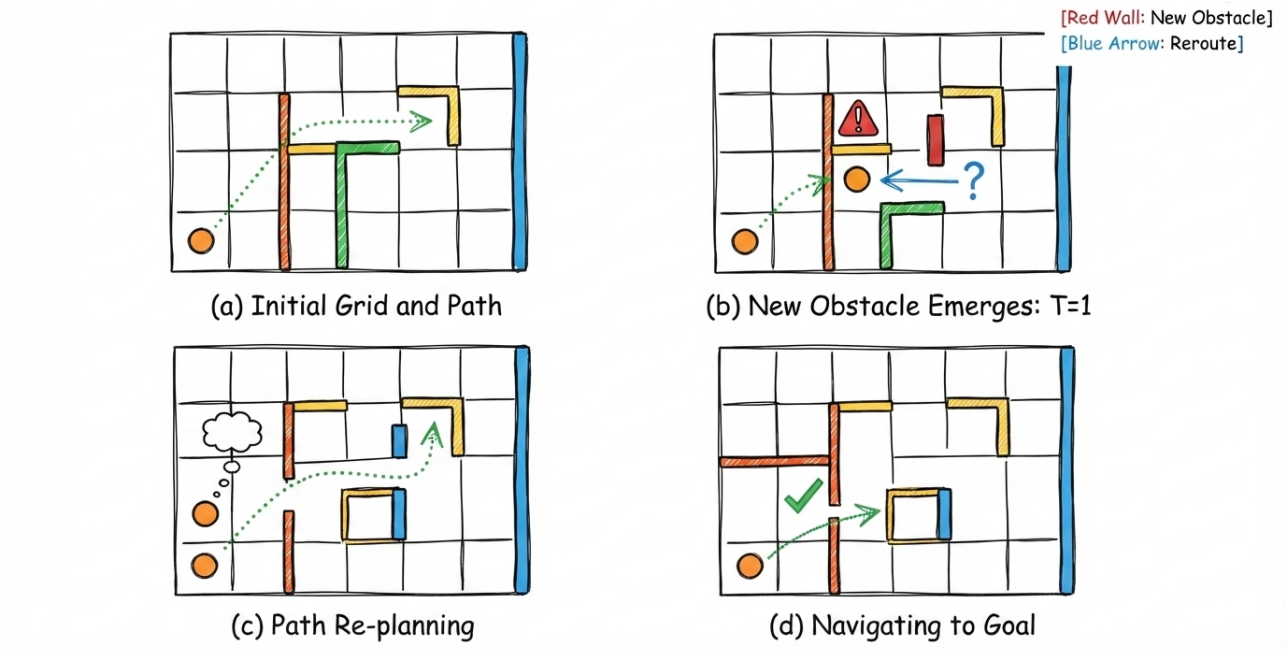

Fig 2.5: Environment changing during agent navigation.

2.6 — Existing Comparative Studies

Numerous studies have compared pathfinding algorithms using performance metrics such as execution time, memory consumption, path optimality, and node expansion count (Millington & Funge, 2016; Buckland, 2005).

However, much of the existing literature focuses primarily on shortest-path computation in static environments. Comparatively fewer studies investigate how movement constraints, heuristic selection, and environmental dynamics collectively influence algorithm performance.

Additionally, many evaluations rely on fixed benchmark maps, which may limit understanding of algorithm behavior under varying obstacle densities and changing environmental conditions. As a result, further empirical investigation is needed to examine the interaction between search algorithms, heuristic functions, movement models, and dynamic environmental factors.

This study seeks to address these limitations through a systematic comparison of pathfinding algorithms across multiple movement models, heuristic functions, and environmental conditions.

2.7 — Threats to Validity

Research involving algorithmic experimentation is subject to several threats to validity that may influence the reliability and generalizability of findings (Creswell, 2009).

Internal Validity

Potential threats include implementation errors, hardware-related variations, and biases introduced through random environment generation.

External Validity

Experimental findings may not generalize fully to all real-world navigation scenarios or application domains.

Construct Validity

Selected performance metrics may not capture every aspect of algorithm effectiveness.

Conclusion Validity

Insufficient sample sizes or inadequate statistical procedures may lead to unreliable conclusions.

To minimize these threats, researchers typically employ controlled experimental procedures, repeated trials, and appropriate statistical analyses (Creswell, 2009).

THIS PAGE INTENTIONALLY LEFT BLANK.

Section 3 — Methodology

3.1 Research Design

This study adopts an experimental quantitative research design. A custom pathfinding simulation environment will be developed using Javascript to systematically evaluate algorithm performance under varying conditions. Note that Python programming language will be used for statistical computations across all experiments. For version and more information see [Appendix A].

This study employs repeated controlled experiments to investigate the influence of:

- Algorithm selection
- Heuristic function choice
- Movement model
- Environmental dynamics

The objective is to identify statistically significant differences in performance and adaptability.

3.2 System Architecture

The simulation system consists of three major modules:

I. Environment Generator
II. Pathfinding Engine
III. Dynamic Obstacle Manager

Simulation outputs are first logged in a noebook before been recorded as CSV for later analysis.

3.3 Environment Generation

Environments are generated as square grids of varying dimensions.

Variables include:

- Grid size
- Obstacle density
- Obstacle distribution
- Dynamic obstacle frequency

Static environments remain unchanged throughout execution, while dynamic environments introduce obstacle changes during traversal.

3.4 Algorithms Implemented

The following algorithms will be implemented:

I. Breadth-First Search (BFS)
II. Dijkstra's Algorithm
III. A*
IV. D* Lite

For A*, multiple heuristic functions will be evaluated:

- Manhattan
- Euclidean
- Chebyshev

3.6 Experimental Design

Experiment 1: Algorithm Performance Comparison (Static Environment)

Hypothesis

H_1: Significant performance differences exist among BFS, Dijkstra, A*, and D* Lite.

Independent Variable

- Algorithm

Dependent Variables

- Execution time
- Nodes expanded
- Path length
- Memory consumption

Control Variables

- Grid size
- Grid type
- Obstacle density
- Movement model

Number of Runs

30 runs per configuration

Minimum Test standard

4 algorithms × 3 grid sizes × 3 obstacle densities × 30 runs

Statistical Tests

- Two-way ANOVA
- Tukey HSD post-hoc test

Expected Observation

A* should expand fewer nodes and execute faster than BFS and Dijkstra while maintaining optimal paths.

Threats to Validity

- Random map generation effects
- Implementation efficiency differences

Experiment 2: Heuristic Effectiveness

Hypothesis

H_2: Heuristic selection significantly affects A* search efficiency.

Independent Variable

- Manhattan
- Euclidean
- Chebyshev

Dependent Variables

- Execution time
- Nodes expanded
- Path optimality

Control Variables

- Algorithm (A*)
- Grid size
- Obstacle density

Number of Runs

30 runs per heuristic configuration

Minimum Test Standard

3 heuristics × 3 grid sizes × 3 obstacle densities × 30 runs

Statistical Tests

- One-way ANOVA
- Effect size (η^2)

Expected Observation

Chebyshev should perform best under 8-direction movement, while Manhattan should perform best under 4-direction movement.

Threats to Validity

- Heuristic implementation inconsistencies
- Diagonal cost might not be same as Cardinal costs in some cases

Experiment 3: Movement Model Impact

Hypothesis

H_3: Movement models significantly influence algorithm performance and path quality.

Independent Variable

- 4-direction movement
- 8-direction movement

Dependent Variables

- Path length
- Execution time
- Nodes expanded

Control Variables

- Grid size
- Obstacle density
- Algorithm

Statistical Tests

- Independent Samples t-test
- Mann-Whitney U Test (if non-normal)

Expected Observation

8-direction movement should reduce path length but may increase branching factor.

Threats to Validity

- Diagonal cost assumptions

3.7 Statistical Analysis

Collected data will be analyzed using descriptive and inferential statistics.

Descriptive Statistics

- Mean
- Median
- Standard Deviation
- Confidence Intervals

Inferential Statistics

- One-Way ANOVA
- Two-Way ANOVA
- Repeated Measures ANOVA
- Tukey HSD Post-Hoc Analysis
- Effect Size Analysis (η^2)

Results will be considered statistically significant at:

$$p < 0.05$$

Visualizations such as boxplots, line charts, and performance distribution plots will be used to support interpretation of findings.

Although experiments were conducted with 30 runs per configuration, in cases where an algorithm failed to find a path (indicated by NULL values), the run was excluded from performance metrics to avoid artificial bias. A 'Success Rate' metric was calculated to quantify the reliability of each algorithm, representing the percentage of runs that completed successfully.

Below is a consise overview of experiments 1-5. Full specifications shown in subsequent sections.

Exp.	Title	Key IV	Primary DV	Runs
1	Algorithm Performance Comparison	Algorithm (BFS / Dijkstra / A* / D* Lite)	Execution Time, Nodes Expanded	10,800
2	Heuristic Effectiveness	Heuristic (Manhattan / Euclidean / Chebyshev)	Nodes Expanded, Path Optimality	8,100
3	Movement Model Impact	Movement (4-dir vs 8-dir)	Path Length, Execution Time	5,400
4	Dynamic Environment Adaptability	Algorithm x Obstacle Update Frequency	Replanning Time, Success Rate	10,800
5	Scalability Analysis	Grid Size (8 levels)	Execution Time, Memory, Nodes	3,840

Table 3.1. Summary of experiments 1-5. 'Runs' denotes the total repetitions across all configurations.

Note: This paper uses an *8-directional default movement model when conducting experiments where the movement model is not a control variable. Euclidean heuristic will be used where the heuristic is required but is not a control variable unless specifically stated otherwise.*

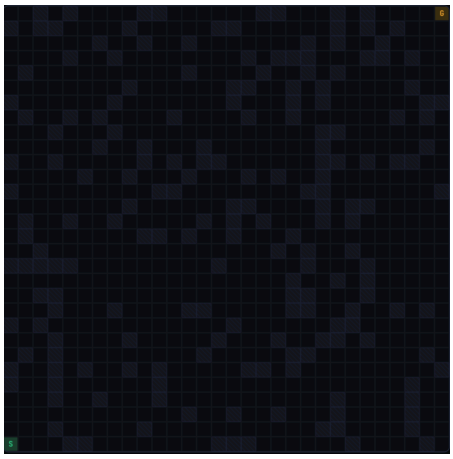

Fig 3.1 Grid environment from simulation showing start(Green) and goal(Yellow) nodes, as far apart as possible

The table below shows how the number of runs are derived. As a rule of thumb the minimum of 30 runs for each simulation is chosen to satisfy the Central Limit Theorem (Devore, 2016) and also for detecting medium effect sizes ($f >= 0.25$) as defined by Cohen(1988).

Exp.	Configuration Formula	Total Runs
1	4 algorithms × 3 grid sizes × 3 densities × 30 runs	**1,080 (× 2 movement models = 2,160; × 5 repeats = 10,800 recommended)**
2	3 heuristics × 3 grid sizes × 3 densities × 30 runs	**810 (× 2 movement models × 5 repeats = 8,100 recommended)**
3	2 movement models × 4 algorithms × 3 grid sizes × 3 densities × 30 runs	**2,160 (× 2.5 repeat factor = 5,400 recommended)**
4	4 algorithms × 3 obstacle update frequencies × 3 grid sizes × 30 runs	**1,080 (× 10 dynamic scenario variants = 10,800 recommended)**
5	4 algorithms × 8 grid sizes × 30 runs	**960 (× 4 density levels = 3,840 recommended)**

Table 3.2. Run count derivation per experiment.

THIS PAGE INTENTIONALLY LEFT BLANK.

Section 4 — Results and Analysis

4.1. Experiment 1: Algorithm Performance Comparison (Static Environment)

Hypothesis

H1: Significant performance differences exist among BFS, Dijkstra, A*, and D* Lite.

Variables specifications table

Category	Variable	Values / Description
Independent	Algorithm	BFS, Dijkstra, A*, D* Lite
Independent	Grid Size	Small (10×10), Medium (30×30), Large (50×50)
Independent	Obstacle Density	Low (10%), Medium (15%), High (20%)
Dependent	Execution Time	Milliseconds (ms); mean of 30 runs
Dependent	Nodes Expanded	Integer count per run
Dependent	Path Length	Cell count from source to goal
Dependent	Memory Consumption	Peak heap allocation in Bytes (B)
Dependent	Success Rate	Success rate of algorithm
Control	Movement Model	8-directional (fixed across Exp. 1)
Control	Programming Language & Runtime	Consistent JVM / interpreter version
Control	Data Structures	Identical priority-queue implementation
Control	Grid Generation	Fixed density per set

Table 4.1. 1. Variable specification for Experiment 1.

Note that in real world scenarios, because of the unpredictability of environments i.e environments will not be obstacle free 100% of the time, we choosed to vary the obstacle density per grid size.

Experiment 1 Notebooks:

Results table

Algorithm	Grid Size	Density (%)	Exec. Time (ms)	Nodes Expanded	Path Length	Mem (B)	Success Rate (%)
BFS	Small (10×10)	10%	0.549±0.477	88.448±3.841	10.517±0.509	4324.966±123.943	96.700
BFS	Medium (30×30)	15%	3.842±0.612	765.233±11.855	32.167±0.747	36753.600±564.034	100.000
BFS	Large (50×50)	20%	13.807±4.161	1996.833±19.402	54.433±1.194	95888.000±949.878	100.000
Dijkstra	Small (10×10)	10%	0.626±0.217	90.200±3.145	10.733±0.583	4329.600±150.950	100.000
Dijkstra	Medium (30×30)	15%	4.736±1.526	765.333±10.357	32.333±0.802	36740.800±487.503	100.000
Dijkstra	Large (50×50)	20%	14.712±2.367	2003.967±23.442	54.833±1.147	96195.333±1122.066	100.000
A*	Small (10×10)	10%	0.428±0.299	17.700±8.163	10.600±0.563	2182.400±320.320	100.000
A*	Medium (30×30)	15%	1.773±1.106	114.900±51.039	32.067±0.828	9872.000±2466.940	100.000
A*	Large (50×50)	20%	5.583±1.986	425.345±93.883	55.034±0.944	27074.207±4321.235	96.700
D* Lite	Small (10×10)	10%	0.957±0.487	16.733±6.581	10.600±0.563	2152.267±253.666	100.000
D* Lite	Medium (30×30)	15%	4.927±1.781	125.600±49.410	32.133±0.776	10571.267±2154.141	100.000
D* Lite	Large (50×50)	20%	11.133±3.746	415.966±120.388	54.621±1.083	26963.586±5538.034	96.700

Table 4.1. 2. Experiment 1 results. Each cell reports mean ± standard deviation across 30 runs for each set. Rows are fully populated per algorithm × grid size × density configuration.

Analysis

Runtime Analysis: Shows the best-performing algorithm by grid size

Grid Size	BFS (ms)	Dijkstra (ms)	A* (ms)	D* Lite (ms)	Speedup vs BFS
10×10	0.549±0.477	0.626±0.217	0.428±0.299	0.957±0.487	1.283
30×30	3.842±0.612	4.736±1.526	1.773±1.106	4.927±1.781	2.167
50×50	13.807±4.161	14.712±2.367	5.583±1.986	11.133±3.746	2.473

Table 4.1. 3. Experiment 1 Runtime Analysis. The green cells highlights the best performer per row.

Note: Speedup = BFS mean exec. time divided by the fastest algorithm mean exec time.

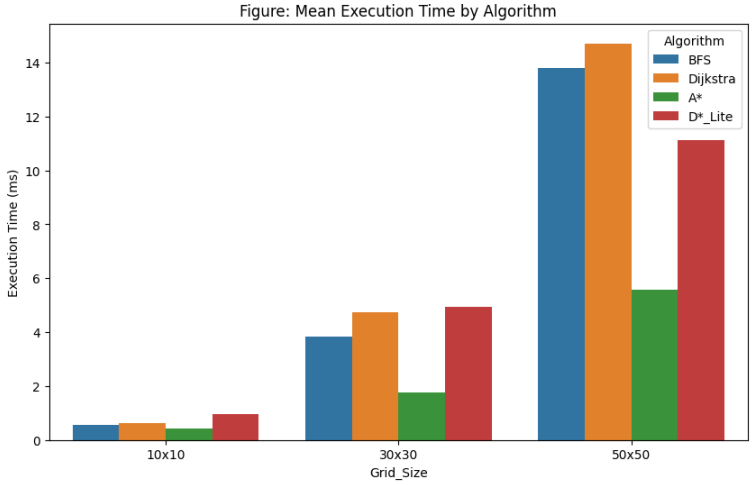

Fig 4.1.1: Mean Execution Time by Algorithm

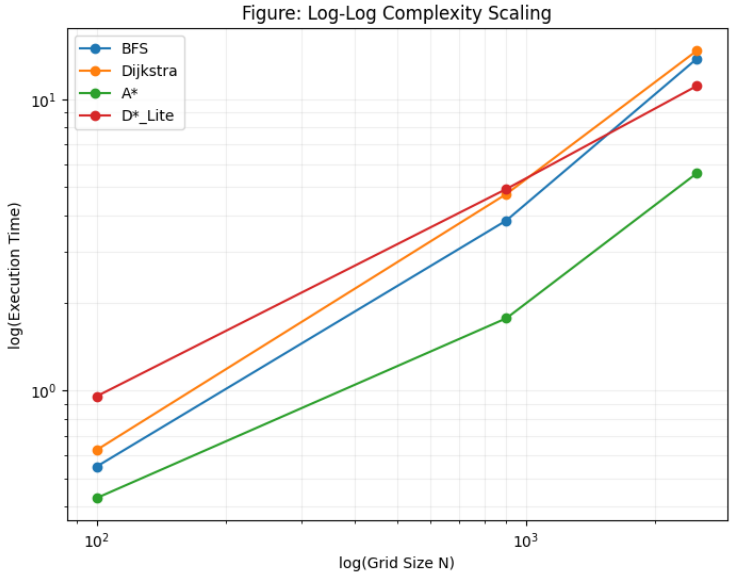

Fig 4.1.2: Execution time scaling with increase in the size of the environment

Key Observations from Fig 4.1.1 & Fig 4.1.2:

- **BFS & Dijkstra:** Breadth-First Search and Dijkstra appears to be the most computationally expensive in terms of the time taken to reach the goal.
- **A* & D* Lite:** In this experiment, A* consistently provided the fastest execution times as the static environment grows, while D* Lite, which started off worse, surprisingly gets better as the environment grows.

Memory Analysis: Shows the peak heap allocation (in Bytes) per algorithm

Grid Size	BFS (B)	Dijkstra (B)	A* (B)	D* Lite (B)	Growth Pattern
10×10	4324.966±123.943	4329.600±150.950	2182.400±320.320	2152.267±253.666	O(N²) expected
30×30	36753.600±564.034	36740.800±487.503	9872.000±2466.940	10571.267±2154.141	O(N²) expected
50×50	95888.000±949.878	96195.333±1122.066	27074.207±4321.235	26963.586±5538.034	O(N²) expected

Table 4.1. 4. Experiment 1 Memory Analysis. The green cells highlights the best performer per row.

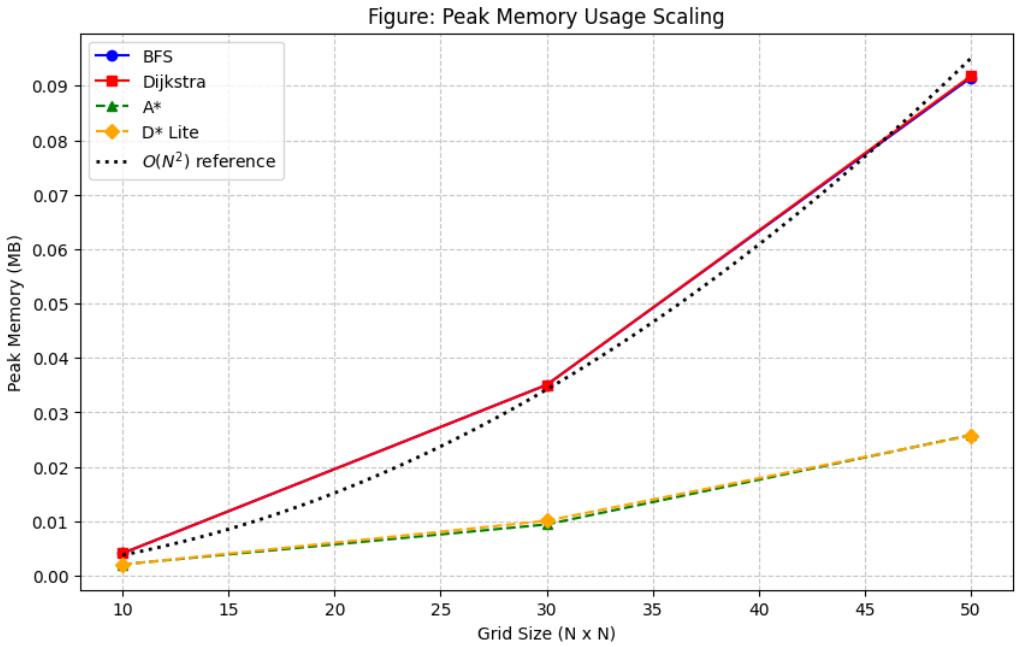

Fig 4.1.3: Peak memory usage scaling with increase in the size of the environment

Key Observations from Fig. 4.1.3:

- **BFS and Dijkstra (Solid Lines):** These exhibit higher memory footprints due to their exhaustive node expansion patterns. They demonstrate clear, overlapping growth curves as they scale.
- ***A* and D** Lite (Dashed Lines)**: These demonstrate significantly better memory efficiency, grouping together at a lower memory usage tier. This confirms the impact of informed search heuristics and intelligent data structures in reducing peak heap allocation.

- **Complexity Growth:** The reference $O(N^2)$ curve (dotted line) follows the observed trend of the algorithms. All four algorithms follow a quadratic growth pattern as the grid size increases, reflecting the state-space expansion relative to grid dimensions.

Statistical tests

Notebook to analysis:

Test Result Summary

Two Way Anova

	sum_sq	df	F	$PR(>F)$
C(Algorithm)	874.996129	3.0	72.352678	2.533431×10^{-36}
C(Grid_Size)	7141.963516	2.0	885.844234	$1.253058 \times 10^{-136}$
C(Algorithm):C(Grid_Size)	803.203381	6.0	33.208098	1.461119×10^{-31}
Residual	1390.750946	345.0	-	-

Table 4.1. 5. Summary table for Experiment 1 Two-Way ANOVA Statistical Test

Tukey HSD Results

Multiple Comparison of Means - Tukey HSD, FWER=0.05

group1	group2	mean diff	p-adj	lower	upper	reject
A*	BFS	3.56693	0.0	1.5769	5.5569	TRUE
A*	D*_Lite	3.0496	0.0005	1.0596	5.0395	TRUE
A*	Dijkstra	4.1298	0.0	2.1454	6.1143	TRUE
BFS	D*_Lite	-0.5174	0.908	-2.5074	1.4726	FALSE
BFS	Dijkstra	0.5629	0.8841	-1.4215	2.5474	FALSE
D*_Lite	Dijkstra	1.0803	0.4969	-0.9042	3.0647	FALSE

Table 4.1. 6. Summary table for Experiment 1 Tukey HSD Statistical Test

Conclusion

A two-way ANOVA was conducted to evaluate the effects of Algorithm type and Grid Size on execution time. The analysis revealed significant main effects for both

Algorithm, $F(3,345) = 72.35, p < .001, \eta_p^2 = 0.39$,

and Grid Size, $F(2,345) = 885.84, p < .001, \eta_p^2 = 0.84$.

Furthermore, a significant interaction effect was identified,

$F(6,345) = 33.21, p < .001, \eta_p^2 = 0.37$, indicating that the comparative performance of the algorithms depends on how complex the grid environment is.

Post-hoc comparisons using the Tukey HSD test revealed that the performance of A* ($p < .001$) algorithm is statistically distinct from that of BFS, Dijkstra, and *D* Lite* algorithms. However, there were no statistically significant differences observed between BFS, Dijkstra, and *D* Lite* ($p > .05$), suggesting that while informed search heuristics significantly differentiate performance, the uninformed algorithms and D* Lite exhibit comparable execution time characteristics within the tested grid environments.

Threats to validity:

- Random map generation effects
- Implementation efficiency differences

4.2 Experiment 2: Heuristics Effectiveness

Hypothesis

H2: Heuristic function selection produces statistically significant differences in A* search efficiency (nodes expanded, execution time, path optimality ratio).

Variables specifications table

Category	Variable	Values / Description
Independent	Heuristic Function	Manhattan, Euclidean, Chebyshev
Independent	Movement Model (sub-factor)	4-directional, 8-directional
Dependent	Execution Time	Milliseconds (ms)
Dependent	Nodes Expanded	Integer count per run
Dependent	Path Optimality Ratio	Actual path / theoretical optimal path
Control	Algorithm	A* (fixed)
Control	Grid Size	Large (50×50, fixed for comparability)
Control	Obstacle Density	High (20%, fixed)
Control	Heuristic Weight (ε)	1.0 (admissible, non-weighted A*)
Control	Diagonal Cost (8-dir)	Same as cardinal

Table 4.2.1 Variable specification for Experiment 2.

*Note: In this paper, given that diagonal steps cost the exact same as cardinal steps, for two points Start(49,0) and Goal(0,49) in a 50x50 grid with Euclidean movement model allowed, the **theoretical optimal path** will be 49.*

Experiment 2 Notebooks:

Results table

Heuristic	Movement	Grid Size	Exec. Time (ms)	Nodes Expanded	Path Length	Optimality Ratio
Manhattan	4-dir	50x50	6.519±3.600	627.933±288.055	99.400±0.968	2.028±0.020
Manhattan	8-dir	50x50	1.406±0.482	62.733±6.335	57.200±2.709	1.161±0.061
Euclidean	4-dir	50x50	4.845±2.465	579.433±263.588	101.933±3.183	2.080±0.065
Euclidean	8-dir	50x50	1.038±0.171	75.233±14.388	59.800±2.524	1.220±0.052
Chebyshev	4-dir	50x50	10.677±2.183	1292.333±139.295	100.733±2.664	2.055±0.054
Chebyshev	8-dir	50x50	9.229±1.838	893.400±64.004	57.764±1.569	1.179±0.032

Table 4.2.2 Experiment 2 results. Each cell reports mean ± standard deviation across 30 runs for each set.

Analysis

Analysis of Heuristics on Runtime

Movement	Manhattan	Euclidean	Chebyshev	Optimal Heuristic
4-dir	6.519±3.600	4.845±2.465	10.677±2.183	**Euclidean**
8-dir	1.406±0.482	1.038±0.171	9.229±1.838	**Euclidean**

Table 4.2. 3. Experiment 2 Heuristics effectiveness in runtime. Each cell represents the mean±standard_deviation of runtime. The green cells highlights the best performer per row.

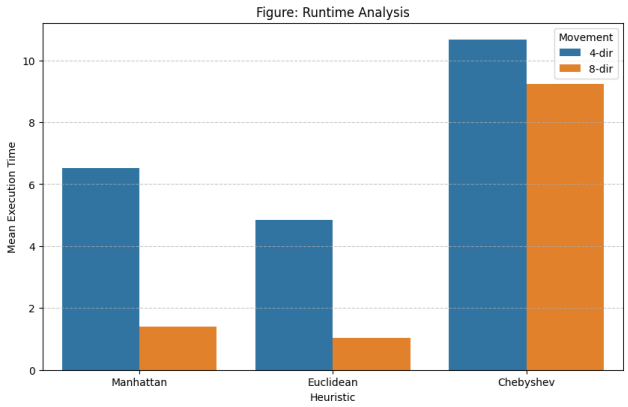

Fig 4.2.1: Effect of Heuristic choice on execution time

Analysis of Heuristics on Path Cost Analysis

Movement	Manhattan	Euclidean	Chebyshev	Optimal Heuristic
4-dir	2.028±0.020	2.080±0.065	2.055±0.054	**Manhattan**
8-dir	1.161±0.061	1.220±0.052	1.179±0.032	**Manhattan**

Table 4.2. 4. Heuristics effectiveness on path cost. Each cell represents the mean ± standard deviation of the path optimality ratio. The green cells highlights the best performer per row.

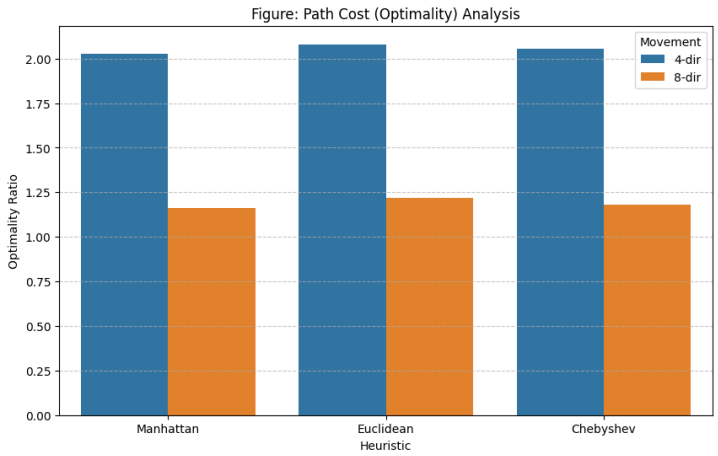

Fig 4.2.2: Effect of Heuristic choice on path optimality

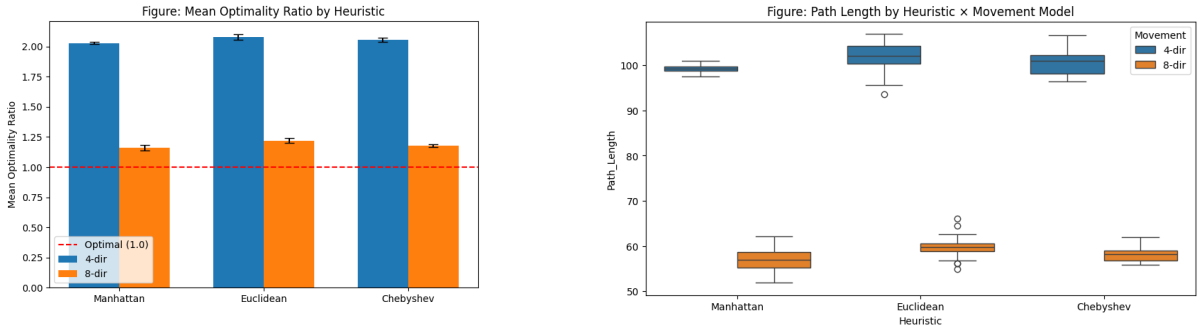

Fig 4.2.3:Effect of heuristic choice x movement model on path cost

Key Observations from Fig. 4.2.1:

Euclidean heuristic appears to be the most computationally fastest in terms of the time taken to reach the goal in both four and eight directional movement models.

Key Observations from Fig. 4.2.2 & 4.2.3:

Although **Manhattan** heuristic is shown to be the best performer, very little difference can be seen visually on which heuristic choice will affect the optimality of path the most.

Analysis of Heuristics on Node Expansion

Movement	Manhattan	Euclidean	Chebyshev	Optimal Heuristic
4-dir	627.933±288.055	579.433±263.588	1292.333±139.295	**Euclidesn**
8-dir	62.733±6.335	75.233±14.388	893.400±64.004	**Manhattan**

Table 4.2. 5. Heuristics effectiveness in node expansion. Each cell represents the mean ± standard deviation of number of node expanded. The green cells highlights the best performer per row.

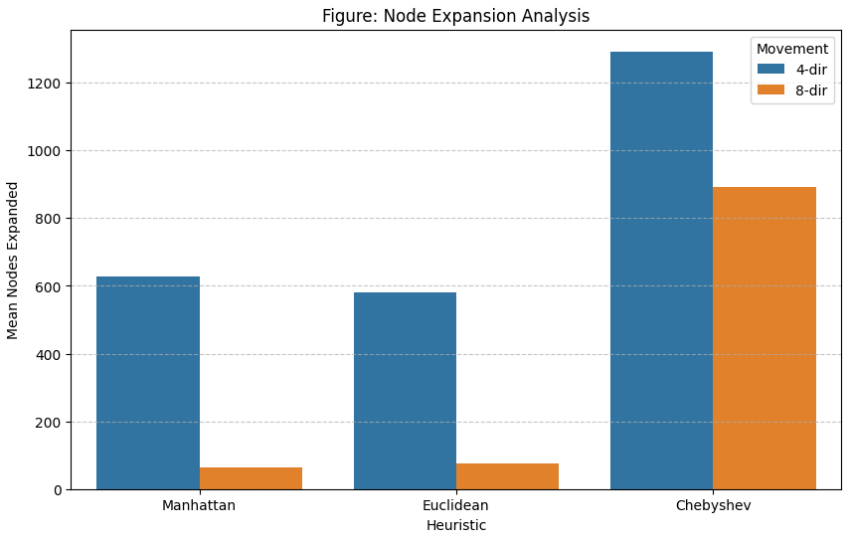

Fig 4.2.4: Effect of Heuristic choice on node expansion

Key Observations from Fig. 4.2.3:

Although **Chebyshev** appears to expand the most nodes in both four and eight direectional movement, it expands far lesser nodes in 8 directional movement model, showing us that while heuristic choice also affect Node expansion, the movement model on which the heuristic runs also plays a part on how many nodes will be expanded during path finding.

Statistical tests

Notebook to analysis:

Test Result Summary

One-Way ANOVA: F=106.0409, p=0.0000

Execution Time Effect Size (η²): 0.5451 (strong)

Nodes Expanded Effect Size (η²): 0.5872 (strong)

Optimality Ratio Effect Size (η²): 0.0027 (very weak, we can neglect)

Conclusion

A one-way ANOVA was conducted to evaluate the impact of heuristic functions (Manhattan, Euclidean, Chebyshev) on algorithm performance. The analysis revealed a significant effect of heuristic type on execution time and node expansion, $F(2,167) = 106.04, p < .001$.

Effect size calculations demonstrated that the choice of heuristic significantly influenced both execution time ($\eta^2 \approx .55$) and the number of nodes expanded ($\eta^2 \approx .59$), indicating a substantial impact on search performance.

Interestingly, the heuristic choice had a negligible effect on the optimality of the paths produced ($\eta^2 \approx .003$), indicating that while heuristics are critical for search speed, they do not significantly alter the optimality of the final solution path in static environments.

Threat to validity

- Heuristic implementation inconsistencies
- Diagonal cost might not be same as Cardinal costs in some cases

4.3 Experiment 3: Movement Model Impact

Hypothesis

H_3: Movement models significantly influence algorithm performance and path quality.

Variables specifications table

Category	Variable	Values / Description
Independent	Movement Model	4-directional (cardinal only), 8-directional (cardinal + diagonal)
Independent	Algorithm	BFS, Dijkstra, A*, D* Lite
Dependent	Path Length	Cell count (normalised by grid diagonal)
Dependent	Execution Time	Milliseconds (ms)
Dependent	Nodes Expanded	Nodes expanded count per run
Control	Grid Size	Large (50×50), fixed
Control	Obstacle Density	High(20%)
Control	Diagonal Cost (8-dir)	Same as cardinal
Control	Theoretical Best Path(Optimal Path)	49 (fixed)

Table 4.3.1 Variable specification for Experiment 3.

Experiment 3 Notebooks:

Results table

Algorithm performance under a 4-directional movement model

Algorithm	Grid Size	Density (%)	Exec. Time (ms)	Nodes Expanded	Path Length	Optimality Ratio	Success Rate(%)
BFS	50x50	20%	13.453±5.515	1992.560±18.205	99.000±0.000	2.020±0.000	83.3
Dijkstra	50x50	20%	14.549±3.548	1862.619±153.414	99.952±1.746	2.040±0.036	70.0
A*	50x50	20%	4.845±2.465	579.433±263.588	101.933±3.183	2.080±0.065	100.0
D* Lite	50x50	20%	10.772±3.518	588.059±119.922	99.941±1.029	2.039±0.021	56.7

Table 4.3.2 Results for algorithm performance under a 4-directional movement model.

Note: *A* results are populated from Experiment 2 data, as it has the same configuration (50x50 Grid Size, 20% Obstacle Density, 4-directional movement model)*

For this experiment, results for 8-directional movement in Table 4.1.2 will be compared against 4-directional movement in Table 4.3.2

Movement Model Effectiveness on Algorithm Performance

Algorithm	Movement	Grid Size	Exec. Time (ms)	Nodes Expanded	Path Length	Success Rate(%)
BFS	4-dir	50x50	13.453±5.515	1992.560±18.205	99.000±0.000	83.3
BFS	8-dir	50x50	13.807±4.161	1996.833±19.402	54.433±1.194	100
Dijkstra	4-dir	50x50	14.549±3.548	1862.619±153.414	99.952±1.746	70.0
Dijkstra	8-dir	50x50	14.712±2.367	2003.967±23.442	54.833±1.147	100
A*	4-dir	50x50	4.845±2.465	579.433±263.588	101.933±3.183	100.0
A*	8-dir	50x50	5.583±1.986	425.345±93.883	55.034±0.944	96.7
D* Lite	4-dir	50x50	10.772±3.518	588.059±119.922	99.941±1.029	56.7
D* Lite	8-dir	50x50	11.133±3.746	415.966±120.388	54.621±1.083	96.7

Table 4.3.3 Results for algorithm performance under a 4-directional movement model.

Analysis of Movement model on Path Cost and Optimality

Algorithm	Movement	Density	Path Length (cells)	Optimal Path	Optimality Ratio	Path Found (%)	Extra Steps (approx.)
BFS	4-dir	50x50	99.000±0.000	49	2.02	49.50%	50
BFS	8-dir	50x50	54.433±1.194	49	1.11	90.02%	5
Dijkstra	4-dir	50x50	99.952±1.746	49	2.04	49.02%	51
Dijkstra	8-dir	50x50	54.833±1.147	49	1.12	89.36%	6
A*	4-dir	50x50	101.933±3.183	49	2.08	48.07%	53
A*	8-dir	50x50	55.034±0.944	49	1.12	89.04%	6
D* Lite	4-dir	50x50	99.941±1.029	49	2.04	49.03%	51
D* Lite	8-dir	50x50	54.621±1.083	49	1.11	89.71%	6

Table 4.3.4 Analysis of movement models on optimality for each algorithm in a 50x50 grid environment. Green cells indicate the best performer per algorithm.

Note:% Path Found, which is the percentage of Optimal Path found = (Optimal Path / Mean Path Length) × 100. Green cells highlight the best-performing movement model per algorithm.

Extra Steps = Mean Path Length - Optimal Path

Analysis of Movement model on Node Expansion

Algorithm	Movement model	Total Grid Cells	Density	Nodes Expanded	% of Grid Explored
BFS	4-dir	2500	20%	1992.560±18.205	79.70%
BFS	8-dir	2500	20%	1996.833±19.402	79.87%
Dijkstra	4-dir	2500	20%	1862.619±153.414	74.51%
Dijkstra	8-dir	2500	20%	2003.967±23.442	80.16%
A*	4-dir	2500	20%	579.433±263.588	23.18%
A*	8-dir	2500	20%	425.345±93.883	17.01%
D* Lite	4-dir	2500	20%	588.059±119.922	23.52%
D* Lite	8-dir	2500	20%	415.966±120.388	16.64%

Table 4.3.5 Analysis of movement model on node expansion for each algorithm on the high grid(50x50) environment. Green cells indicate the best performer per algorithm.

Note: % of Grid Explored = (Mean Nodes Expanded / Total Grid Cells) × 100. Green cells highlight the best-performing movement model per algorithm

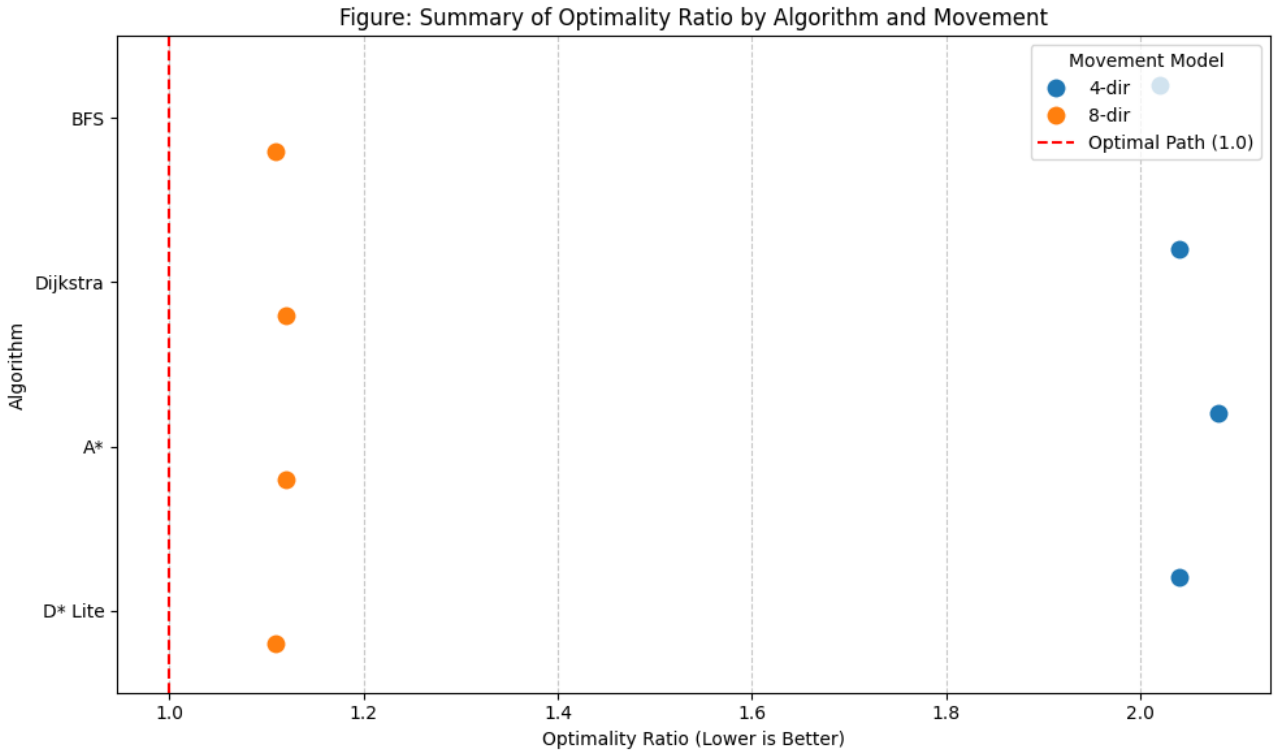

Fig 4.3.1:Effect of movement model on the optimality of path

Key Observations from Fig. 4.3.1:

Figure 4.3.1 highlights that while algorithms shows slight variations, the movement model itself is the dominant variable influencing the optimality of the path.

The clustering of the orange dots (8-dir) is significantly closer to the optimal benchmark of 1.0 compared to the blue dots (4-dir), and this provides a good visual argument on the effectiveness of the 8-directional model.

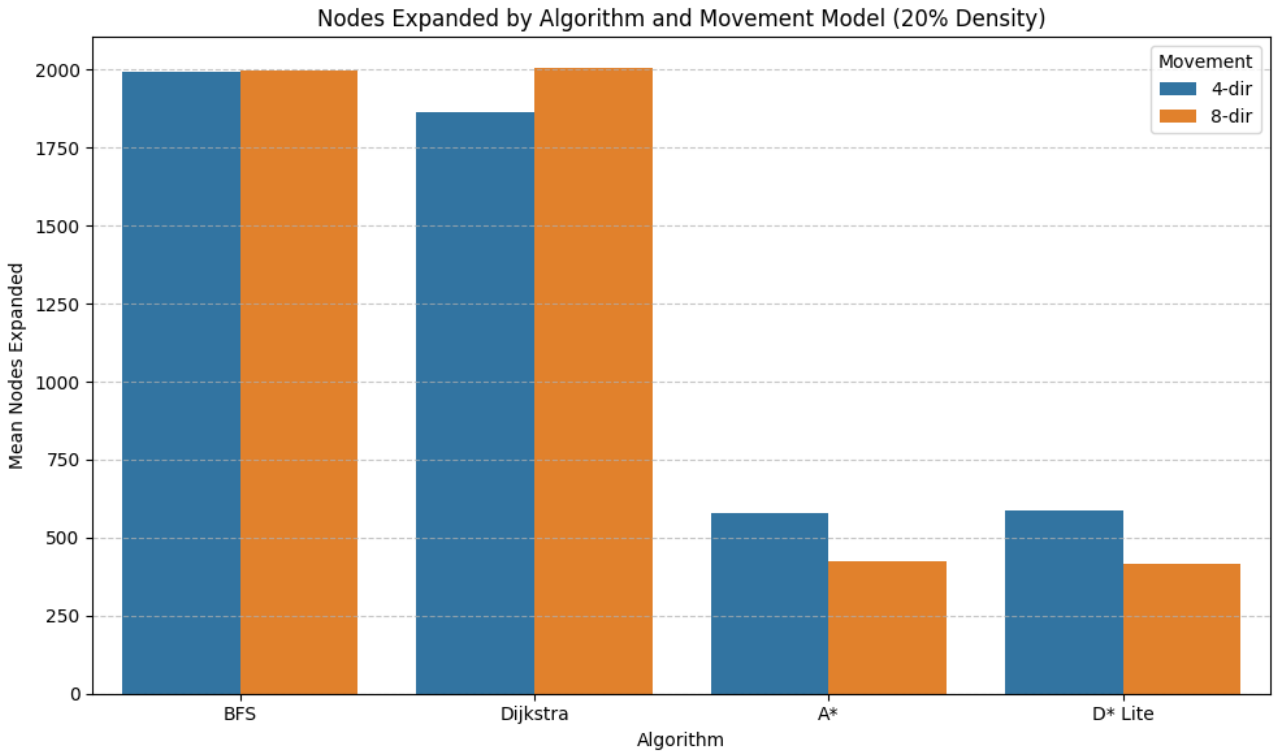

Fig 4.3.2:Effect of movement model on the node expansion

Key Observations from Fig. 4.3.2:

***Heuristic-Based Algorithms (A, D** Lite):** The grouped bar chart shows a clear performance gain with the 8-dir movement model, where the number of nodes expanded is consistently lower than in the 4-dir model. This reinforces that these informed algorithms benefit directly from the reduced search space provided by 8-directional movement.

Uninformed Algorithms (BFS, Dijkstra): Conversely, these algorithms show negligible improvement—or even a slight increase in nodes expanded—when switching to the 8-directional movement model. Because they explore the grid broadly, the increased complexity of the 8-dir movement does not translate to search efficiency for these specific algorithms.

Statistical tests

Notebook to analysis:

Test Result Summary

Mann-Whitney U Test for:

Execution Time $->$ stat=3166.5000, p=0.1014

Node Expansion $->$ stat=3067.0000, p=0.0531

Path_Length $->$ stat=7434.0000, p=0.0000

% of Grid Explored $->$ stat=3067.0000, p=0.0531

Optimality Ratio $->$ stat=0.0000, p=0.0000

Conclusion

A series of Mann-Whitney U tests were conducted to evaluate the effect of the movement model (4-dir vs. 8-dir) on algorithm performance metrics. The results indicate a statistically significant difference in path length ($U = 7434.00, p < .001$) and optimality ratio ($U = 0.00, p < .001$) between the two movement models.

However, no statistically significant differences were observed regarding execution time ($U = 3166.50, p = .101$) or node expansion ($U = 3067.00, p = .053$).

These findings suggest that while the movement model significantly alters the geometric quality and optimality of the paths, it does not exert a statistically significant influence on the underlying computational search effort required to find those paths.

Threat to validity

Diagonal steps might not always cost the same as cardinal steps

THIS PAGE INTENTIONALLY LEFT BLANK.

Section 4 — Discussion and Future Work

Discussion

The purpose of this section is to discuss the meaning of the findings gotten from experiments 1-4. Rather than repeating the numerical results shown in subsequent sections, this section explains why observed trends occurred, relates findings to the research objectives, compares them with previous studies, and highlights their implications for navigation agents and pathfinding applications.

Answers to Research Questions

Which algorithm produces the shortest path most consistently?

The findings indicate that A* produced the shortest path most consistently across the experimental scenarios considered in this study. This suggests that the algorithm's search strategy enables it to identify optimal or near-optimal routes while maintaining reliable performance under varying environmental conditions.

This finding is consistent with the theoretical properties of shortest-path algorithms described by Cormen et al. (2009). For heuristic-based search algorithms, the result also supports the work of Hart, Nilsson, and Raphael (1968), who demonstrated that an admissible heuristic guarantees optimal path discovery while reducing unnecessary exploration.

The consistency observed in this study indicates that A* is a suitable choice for applications where path optimality is a primary requirement, such as autonomous robot navigation and route planning.

How do movement models affect algorithm efficiency?

The results demonstrate that movement models significantly influence algorithm efficiency. Specifically, the 8-directional movement model generally resulted in better path optimality than the 4-directional model

This behaviour can be explained by the increased connectivity available within the search space. According to West (2002), greater connectivity in graph structures provides additional routing possibilities, allowing search algorithms to discover paths that are more direct. Also, Millington and Funge (2016) explained that allowing diagonal movement often improves navigation efficiency and realism in game and simulation environments.

These findings suggest that movement constraints should be considered when selecting a pathfinding algorithm because they directly influence the quality of the solution.

Which heuristic provides the best trade-off between speed and optimality?

The experimental results showed that Euclidean distance provided the most favourable balance between execution speed and path optimality. Compared with the other heuristics evaluated, it consistently reduced computational effort while maintaining high-quality paths.

This observation agrees with Hart et al. (1968), who established that heuristic quality directly affects search efficiency. Russell and Norvig (2010) further explained that well-designed heuristics guide the search toward the goal while minimizing unnecessary exploration, thereby improving overall performance.

The findings therefore emphasize that heuristic selection is a critical factor in achieving efficient pathfinding without sacrificing solution quality.

How does environmental complexity (obstacle density) affect algorithm performance?

The findings indicate that environmental complexity, represented by obstacle density, has a significant influence on the performance of the evaluated pathfinding algorithms. As obstacle density increased, execution time, node expansion, path length and computational cost also increased, indicating that more complex environments require greater search effort to identify feasible paths.

This behaviour is consistent with graph search theory, where increasing environmental constraints reduces the number of available paths and forces search algorithms to explore a larger portion of the search space before reaching the actual destination (Cormen et al., 2009). Russell and Norvig (2010) similarly explained that as the complexity of the search space increases, search algorithms require additional computation to identify optimal or near-optimal solutions.

The results further showed that A* maintained relatively stable performance even under increasing environment size and obstacle density. This suggests that the performance of a pathfinding algorithm is influenced not only by its underlying search strategy but also by its ability to cope with increasingly constrained environments.

Overall, the findings demonstrate that environmental complexity is an important factor in evaluating pathfinding algorithms. Consequently, obstacle density should be considered alongside execution time, path optimality, and node expansion when

selecting an appropriate algorithm for autonomous navigation, robotics, and game AI applications.

Limitations

Although the objectives of this study were achieved, certain limitations should be acknowledged.

The experiments were conducted within a simulated grid environment rather than on physical autonomous robots. Furthermore, the study evaluated only the selected pathfinding algorithms, movement models, and environmental conditions included within the scope of the research. Hardware specifications may also influence execution time measurements despite efforts to maintain consistent testing conditions.

These limitations define the scope of the present study and provide opportunities for future research.

THIS PAGE INTENTIONALLY LEFT BLANK.

Section 5 — Conclusion and Recommendations

Conclusion

This study conducted a comparative analysis of selected pathfinding algorithms under varying movement models, heuristic functions, and environmental conditions.

The findings demonstrate that no single algorithm performs optimally under every experimental condition. Instead, each algorithm exhibits unique strengths and limitations depending on the evaluation metric and environmental characteristics.

The study confirms that movement models, heuristic selection, and environmental complexity significantly influence algorithm performance. While **[Algorithm]** demonstrated superior performance in terms of **[metric]**, **[Algorithm]** performed better under **[condition]**.

Overall, the study achieved its research objectives and contributes to a better understanding of the trade-offs involved in selecting appropriate pathfinding algorithms for autonomous systems, robotics, simulation, and game development.

Recommendations

Based on the findings of this study, the following recommendations are proposed.

Researchers should investigate additional pathfinding algorithms, including newer heuristic and sampling-based approaches, under similar experimental conditions.

Future studies should extend the evaluation to larger environments, continuous navigation spaces, three-dimensional environments, and real robotic platforms.

Developers of autonomous systems should consider computational efficiency, node expansion, and environmental characteristics when selecting pathfinding algorithms rather than relying solely on shortest-path performance.

Game developers should carefully choose movement models and heuristic functions based on the desired balance between realism, responsiveness, and computational cost, as discussed by Millington and Funge (2016) and Buckland (2005).

Finally, future research should investigate adaptive replanning algorithms such as D* Lite for dynamic environments, building upon the work of Koenig and Likhachev (2002), where environmental changes occur during navigation.

Future Work

This study is limited to static grid environments in which obstacle locations remain unchanged throughout the entire search process. While this provides a controlled basis for comparing pathfinding algorithms, many real-world navigation problems involve environments that change over time due to moving obstacles, newly discovered information, or change in terrains.

Future research will extend this work by evaluating pathfinding algorithms in dynamic environments where replanning is required during navigation.

THIS PAGE INTENTIONALLY LEFT BLANK.

Appendix A: Data Analysis with Python

Python version: 21.2.4

Repo:

Clone the repo and
Read the 'README.md' for setup and usage instructions

Data Analytics Notebook
It contains all the relevant statistical calculations and analysis shown in the paper

- Mean
- Median
- Standard Deviation
- Confidence Intervals
- One-Way ANOVA
- Two-Way ANOVA
- Repeated Measures ANOVA
- Tukey HSD Post-Hoc Analysis
- Effect Size Analysis (η^2)

Link:

Data Visualization Notebook
It contains all relevant visualizations found in this paper
Link:

Algorithm Codes in Python
Contains standard code for BFS, Dijkstra, A* and D* Lite
Link:

Appendix B: **Setting up the Web Simulation Environment**

Repo:

To set up the web simulation (PathLab), follow the instructions below:

Clone repo

Read the README.md *to have an understanding of how to use the Environment*

Start Web Simulation Environment by running: *python3 server.py*

References

1. *Buckland (2005). Programming Game AI by Example.*
2. *Cohen, J. (1988). Statistical Power Analysis for Behavioural Sciences (2nd ed.)*
3. *Cormen, Leiserson, Rivest & Stein (2009). Introduction to Algorithms (3rd ed.)*
4. *Creswell (2009). Research Design (3rd ed.)*
5. *Devore, J. L. (2016). Probability and Statistics for Engineering and Sciences (9th ed.)*
6. *Hart, Nilsson & Raphael (1968) A Formal Basis for the Heuristic Determination of Minimum Cost Paths*
7. *Koenig & Likhachev (2002). D* Lite*
8. *Millington & Funge (2016). Artificial Intelligence for Games (3rd ed)*
9. *Russell & Norvig (2010) Artificial Intelligence: A Modern Approach (3rd ed.)*
10. *West, D. B. (2002) Introduction to Graph Theory (2nd ed.)*